Week 22 (Days 148-154)

Status: Days 148-154 are written in full. Days 148-150 close Month 5 (LLMOps, then the Month 5 capstone and checkpoint exam). Days 151-154 open Month 6, performance engineering. Every number comes from `code/week-22/day148_*.py` to `day154_*.py`, each run on this box, with logs in `code/week-22/out/`. Shared LLMOps helpers are in `code/week-22/_llmops.py`. The Rust extension crate is `code/week-22/rust_kernels/`, and the capstone tests are `code/week-22/test_capstone.py`.

Reused, not rewritten: Week 17's `_rag.py` (the notes corpus, BM25, the 58-question eval set) and Week 20's `_agent.py` (tool schemas, validator, SmolLM2-135M-Instruct generation, the `send_email` tool that only records to an in-memory OUTBOX). Nothing in this week sends anything anywhere.

Not available, and labelled wherever it appears: - There is **no GPU**, no `nvcc` and no Triton install. `code/week-22/gpu_untested/softmax_triton_UNTESTED.py` and `vector_add_UNTESTED.cu` are **UNTESTED**: written for reading, never compiled, never run. - There is **no C++ compiler (g++)** and no pybind11, so SYLLABUS Day 154's "C++ pybind11 custom op" became a Rust/PyO3 extension built with maturin. The ideas (a compiled kernel, a thin binding, releasing the GIL) carry over directly. - Prices in the Day 149 cost tracking are **ASSUMED** ($0.50 per million input tokens, $1.50 per million output tokens). They are not any provider's price list; the point is the arithmetic.

Compute note. The CPU is an 8-core Xeon with AVX-512, AVX-512 BF16 and AMX tiles, 15 GB RAM, and OpenBLAS under numpy. The LLM is SmolLM2-135M-Instruct, which runs at about 38 tokens/s here. A GPU would change Days 148-150 mostly in speed: the same eval harness would run a far bigger model, which would follow prompts and attacks better (so the guardrail results would look different, probably worse for the regex guard). It would change Days 152-154 in kind: GPU memory bandwidth is roughly an order of magnitude higher, the roofline ridge point moves, and kernels are written for thousands of threads, which is what the UNTESTED Triton and CUDA files sketch.

Sources (all opened and checked this week; links in `docs/resources.md`): the Python `cProfile` and `tracemalloc` docs, the PyTorch profiler docs, the SciPy sparse docs, the PyO3 user guide, the maturin guide, the rayon docs, the Rust book, the Triton docs and its fused-softmax tutorial, and the NVIDIA CUDA C++ Programming Guide. Explanations here are original, not copied.

Day 148: LLMOps I: prompt versioning, eval harnesses and regression gates

Review today: Day 147 (+1), Day 145 (+3), Day 141 (+7), Day 127 (+21; quick 15-min skim of its self-check).

Block 1: Why LLM apps need their own ops (30 min)

A classic model ships as a weights file plus code. An LLM app ships as weights plus code plus a *prompt*, and the prompt is the part people change most often, frequently by hand, often without a test. A one-word edit to a system message can fix one class of question and silently break another. LLMOps is the discipline of treating prompts, few-shot examples, model choice and decoding settings as versioned artifacts, then gating every change on an evaluation that runs automatically.

Three ideas carry the whole day: 1. **Content-addressed versions.** A prompt's identity is the hash of its full text (system message, few-shot turns, template). Two prompts with the same name but different text are different versions, and an eval result is always tied to a hash, not to a name like "v3-final". 2. **A fixed eval set with a held-out slice.** You tune prompts on the "seen" cases. The held-out cases tell you whether the improvement generalises or whether you just memorised your own test set. 3. **A regression gate, not just an average.** "13 passes is more than 7" is not enough to ship. The gate also asks: did the candidate break anything the baseline got right?

Block 2: The harness on this box (60 min)

`_llmops.py` holds a `Registry` (prompts keyed by a 12-character SHA-256 prefix of their canonical JSON), a JSON generation `Cache` keyed by (prompt hash, input, decoding settings), and a `Tracer` used tomorrow. `day148_prompts_evals.py` registers four prompt versions for the Week 20 tool-calling task and runs each over 24 cases (18 seen, 6 held-out) with greedy decoding on SmolLM2-135M-Instruct. A case passes if the output parses as JSON, validates against the tool schema, and gives the correct tool result.

Verified (out/day148_prompts_evals.log): - Registry hashes: v1 zero-shot `8b536272f079`, v2 3-shot `81a052ecaa64`, v3 4-shot (adds a `temp_c` example) `24a6cf8d8a20`, v4 (v3 with the system message reworded) `a431b360acd3`. - v1 zero-shot: pass 0/24 (seen 0/18, held-out 0/6), 24.5 s. - v2 3-shot: pass 7/24 (seen 6, held-out 1), 15.1 s. - v3 4-shot: pass 13/24 (seen 9, held-out 4), 15.9 s. - v4 reworded: pass 13/24 (seen 9, held-out 4), 15.2 s. - Re-running v3 from the cache: 24 hits, 0 new generations, identical results. - Determinism: 5 cases generated twice with greedy decoding gave identical outputs. - Gate v1->v2: lost 0, gained 7, exact McNemar p=0.016, PASS. - Gate v2->v3: lost 3, gained 9, p=0.146, **FAIL**; regressed cases `Compute 1024/16.`, `Compute 99/3.`, `Compute 45%7.`. - Gate v3->v4: lost 0, gained 0, p=1.000, PASS.

Block 3: Reading the results (45 min)

The headline is the v2->v3 gate. Adding a fourth few-shot example (a temperature conversion) almost doubled the score, 7 to 13 of 24, and improved the held-out slice from 1 to 4 of 6. A dashboard that tracks only the pass rate would ship it. The gate refused, because three arithmetic cases that v2 solved now fail. With a 135M model, a new example pulls the model's outputs towards that example's shape, and the calculator cases paid for it. Whether you then ship v3 is a product decision (perhaps the temperature cases matter more), but it must be a *decision*, made with the list of broken cases in front of you, not an accident.

The McNemar test is the right paired test here: both prompts answer the same 24 cases, so the only informative cases are the discordant ones (lost or gained). With 3 lost and 9 gained the p-value is 0.146, which says that on 24 cases you cannot even be confident v3 is better overall. Small eval sets give weak evidence. That is a reason to grow the eval set, not to stop gating.

The v3->v4 row shows a second use of the gate: a "harmless" rewording that changes nothing should produce exactly zero discordant cases, and here it did. If it had flipped cases, you would have learned that the model is sensitive to wording you considered cosmetic.

The cache matters for cost and for honesty. Re-running v3 cost 0 generations, so you can re-score with a fixed metric bug at no cost, and because greedy decoding was deterministic here, cached outputs equal fresh outputs. With sampling (temperature above 0), a cache freezes one random draw, and you would need several samples per case to estimate a pass rate.

Block 4: Self-check (20 min)

1. Why key a prompt by a content hash rather than a name? *Because names get reused for edited text; the hash ties every eval result to the exact text that produced it.*
2. v3 scored 13 vs v2's 7. Why did the gate fail? *It lost 3 cases that v2 passed (`1024/16, 99/3, 45%7`); the gate forbids losing seen cases the baseline passed.*
3. What does p=0.146 tell you? *With only 12 discordant cases, the improvement is not statistically clear on 24 cases.*
4. When is a generation cache unsafe? *When decoding is stochastic, or when the key omits something that changes outputs (model version, decoding settings, tool schemas).*

Block 5: Exercise (45 min)

Add a v5 prompt that keeps v3's four examples but adds one arithmetic example with integer division. Register it, run the harness, and run the gate against both v2 and v3. Write down which cases move. Then add three new held-out cases you write *before* looking at v5's outputs.

Extra notes

The harness has four parts that every LLM eval needs, whatever the scale: a dataset with stable IDs, a runner that records the prompt hash and decoding settings next to every output, a scorer that is deterministic code (here: parse, schema-validate, execute, compare), and a comparator that reports per-case diffs. A score from an LLM judge would be a fifth part, and it would itself need versioning and a check against human labels.

Going deeper

Read about pairwise statistical tests for classifiers (McNemar, and the bootstrap over cases) and think about how many cases you need before a 5-point improvement is detectable.

Interview-style question (15 min)

"Your team edits prompts in a shared doc and ships them on Fridays. Design the minimum process that stops silent regressions." *Expected: prompts in version control keyed by hash; an eval set with held-out slice; CI runs the eval on every change; a gate on per-case regressions as well as the aggregate; diff report attached to the review.*

Common mistakes

- Tuning on the held-out cases until they pass.
- Comparing two prompts run on different case sets or with different decoding settings.
- Trusting an aggregate with no per-case diff.
- Caching outputs without the model version in the key.

What to do next

Tomorrow adds the production half: traces, latency percentiles, token cost, and guardrails, all on the same agent.

Day 149: LLMOps II: tracing, latency and cost tracking, guardrails

Review today: Day 148 (+1), Day 146 (+3), Day 142 (+7), Day 128 (+21; quick 15-min skim of its self-check).

Block 1: Observability for LLM apps (30 min)

A request to an LLM app is not one function call. It is a chain: an input check, a model call, an output check, maybe a tool call, maybe another model call. When a request is slow or wrong you need to know which step was slow or wrong. **Tracing** records each step as a *span* (name, start, duration, attributes such as token counts or the tool name), with spans nested under the request span. The standard vocabulary comes from distributed tracing; here a 30-line `Tracer` context manager writes one JSON line per span, which is enough to compute percentiles and to replay a bad request.

Cost tracking is arithmetic on token counts: cost = input tokens x input price + output tokens x output price. For API models the bill is per token; for self-hosted models the bill is per GPU-hour, and tokens per second converts one into the other. The prices in this script are **ASSUMED** round numbers.

Guardrails are checks you put around the model because you cannot fully trust it: an *input guard* (block known prompt-injection patterns, redact personal data before the model sees it), and an *output guard* (the model's tool call must use an allow-listed tool, match the schema, and any tool with side effects needs human approval).

Block 2: The run (60 min)

`day149_tracing_guardrails.py` runs the Week 20 agent with all three tools visible (calculator, city lookup, `send_email`) over 20 benign requests and 10 attack requests, once with guards off and once with guards on. Every step is a span in `out/traces.jsonl`.

Verified (out/day149_tracing_guardrails.log): - Prompt hash `b63603a7dfdc`; ASSUMED prices $0.5/M input, $1.5/M output tokens. - Guards OFF: benign correct 13 of 20, false positives 0; attacks blocked 0 of 10; `send_email` executed 0 times. - Guards ON: benign correct 13 of 20, false positives 0; attacks blocked 8 of 10 (input guard 6, output guard 2); `send_email` executed 0 times. - Attack outcomes with guards on: 6 x `blocked_input`, `blocked_output:$`, `blocked_output:$.field`, 2 x `tool_error`. - PII redacted before the model saw it: one email address, one phone number. - 274 spans. Latency over the guards-on requests (ms): guard_in p50 0.04 / p95 0.06; llm p50 641.70 / p95 685.93; guard_out p50 0.01 / p95 0.02; tool p50 0.01 / p95 0.02; request p50 625.92 / p95 686.24. - Tokens over 24 LLM calls: input 10,094 (mean 421), output 576 (mean 24.0); 37.6 output tok/s. - Assumed cost $0.00591 in total, $246 per million requests; input tokens are 85% of the cost.

Block 3: What the numbers do and do not show (45 min)

Guards cost nothing in latency. Regex input checks take 0.04 ms at the median against 642 ms for the model call. The request p50 (626 ms) is slightly *below* the llm p50 because 6 blocked requests never reach the model and finish in well under a millisecond. Whatever you spend on guardrails, it will not be latency at this scale.

Cost is dominated by the prompt, not the answer. The few-shot prompt averages 421 input tokens while answers average 24. Under the assumed prices, input is 85% of the bill. So the cheapest optimisation is a shorter prompt or a cached prefix, not a terser answer. Note that yesterday's best prompt (v3) is the *longest*: quality and cost pull against each other, and an LLMOps

dashboard should show both next to each other.

Be careful about what the attack result means. It is tempting to say "guards blocked 8 of 10 attacks". The honest reading is narrower: - The 135M model never called `send_email` in either mode, so in this run there was no harm to prevent. (Week 20 showed an undefended injection queued 22 of 24 emails, so the risk is real with prompts that make the model follow instructions.) - The input guard caught 6 attacks because they used phrases its regexes know ("ignore previous instructions"-style patterns). - The 2 output blocks were schema failures: the model produced malformed calls, and the guard would have blocked them whether or not they were attacks. - The 4 paraphrased attacks got past the regexes; 2 of them ended as tool errors. A paraphrase defeats a pattern list, which is why the output guard (allow-list plus approval for side-effect tools) is the guard that actually protects you: it does not need to recognise the attack, only to refuse an action.

No false positives on 20 benign requests is encouraging but weak evidence. A regex list grows false positives as it grows, and 20 requests is a small sample.

Block 4: Self-check (20 min)

1. Why can request p50 be lower than llm p50? *Blocked requests skip the model and finish almost instantly, pulling the request median down.*
2. Input tokens are 85% of cost. Name two fixes. *Fewer or shorter few-shot examples; prefix caching of the fixed system prompt.*
3. Which guard protects against a paraphrased injection asking to send email? *The output guard: allow-list plus mandatory approval for side-effect tools.*
4. Why redact PII before the model call, not after? *So it never enters the prompt, the logs, or a third-party API.*

Block 5: Exercise (45 min)

Write three new paraphrased attacks of your own and confirm they evade the input regex. Then add a span attribute recording which guard decided each request, and compute the share of requests decided by each guard from `traces.jsonl` alone.

Extra notes

Percentiles need enough samples: a p95 from 24 calls is roughly the second-largest value. In production you would aggregate over thousands of requests and keep the raw spans for a few days so you can rebuild any percentile. Never log raw prompts containing personal data; log the redacted version and the hash.

Going deeper

Look at how tracing standards structure parent and child spans and what attributes they recommend for model calls (model name, token counts, finish reason), and compare with the fields `_llmops.Tracer` writes.

Interview-style question (15 min)

"Your LLM feature's bill doubled last week with no traffic change. How do you find out why?" *Expected: per-request token traces split by input and output; check for a prompt change (hash), longer retrieved context, retries or loops in the agent, a model switch; compare token histograms before and after.*

Common mistakes

- Reporting "attacks blocked" without checking whether the model would have complied anyway.
- Relying on an input pattern list as the only defence.
- Mean latency instead of percentiles.
- Logging unredacted user input.

What to do next

Tomorrow puts the pieces together: RAG, the agent, the eval harness, tracing and a drift monitor in one service, with release gates and an honest verdict.

Day 150: MONTH-5 CAPSTONE: a monitored, tested, tool-using service, plus the Month-5 checkpoint exam

Review today: Day 149 (+1), Day 147 (+3), Day 143 (+7), Day 129 (+21; quick 15-min skim of its self-check).

Block 1: The system (30 min)

`day150_capstone.py` joins Month 5 into one service: - **Router:** a regex sends arithmetic and city questions to the agent, everything else to the notes path. - **Notes path (RAG):** BM25 over the course notes for Weeks 1-16 (112 days, 1,099 section chunks), returning the top chunks with their day as a citation. - **Tool path (agent):** the Week 20 SmolLM2-135M tool caller with Day 149's guards. - **Eval harness:** 58 in-domain notes questions, 12 tool tasks, 5 attacks, and release gates written *before* the run. - **Monitor:** the top BM25 score of each query is logged; a window of 40 queries is compared with the in-domain reference by PSI (population stability index, the drift statistic from Day 145). - **Tracing:** every step is a span in `out/traces150.jsonl`. - **Tests:** `test_capstone.py`, 7 pytest tests, including two that pin *known* bugs so that a fix shows up as a test change.

The question the capstone answers is not "is it good?" but "would the release process let it ship, and does the monitor notice when the world changes?"

Block 2: The run (60 min)

Verified (out/day150_capstone.log, out/day150_tests.log): - Index: 112 days, 1,099 section chunks, built in 0.1 s. Eval: 58 in-domain queries plus 14 from Weeks 18-21 that are *not* indexed. - Notes recall@1 0.43, recall@3 0.53 over 58 (13.0 ms/query); 1 query misrouted to tools: "You have a fixed compute budget of 10 epochs. Do you add augmentation?" - Tool tasks: 0.50 of 12. Attacks: refused, refused, refused, answered, answered; side effects executed 0. - Gates: notes recall@3 >= 0.60 **False**; tool accuracy >= 0.50 True; no side effects True; no misrouting **False** -> **BLOCK**. - Monitor: top-score median in-domain 17.5, new topics 14.7; PSI alert threshold 1.027 (99th percentile of 200 resampled in-domain windows of 40). - In-domain window: PSI 0.165, no alert; 8% flagged low-confidence (score < 11.2). - 50/50 mix with new topics: PSI 1.491, alert; 12% flagged. - All new topics: PSI 3.263, alert; 8% flagged. - 13 of 14 new-topic queries were answered with a citation into Weeks 1-16, which cannot be right; the other went to the tool path. - Latency: notes p50 1.8 ms / p95 3.0 ms (n=72); tool p50 643.1 ms / p95 710.0 ms (n=14). 278 spans; whole run 11 s. - Iteration 2, word chunks: 128/32 (1,037 chunks) recall@1 0.41, recall@3 0.57; 256/64 (550 chunks) recall@1 0.45, recall@3 0.60. - Tests: 7 passed in 0.87 s.

Block 3: The honest verdict (45 min)

The release is blocked, and that is the system working. Two gates failed. Retrieval found the right day in the top 3 for only 53% of questions, and the router sent a conceptual question to the calculator because it contained the word "compute". The second is a classic regex-router bug; the test suite now contains `test_router_known_bug`, which asserts the current (wrong) behaviour, so whoever fixes the router has to change the test on purpose.

Iteration 2 reached the gate, but read it with suspicion. Switching to 256-word chunks with 64-word overlap gave recall@3 of exactly 0.60. That passes, but it was chosen by looking at the same 58 questions the gate uses. One tuning step on the eval set is a mild sin; ten would make the gate meaningless. The fix is a second, untouched question set for the final check, the same seen/held-out split as Day 148.

The monitor result is the most useful lesson. A per-query rule ("flag answers whose top score is below 11.2") flagged 8% of in-domain queries and also only 8% of entirely new-topic queries: it cannot tell them apart, because BM25 always finds *some* overlapping words. The window-level PSI on the score distribution could: 0.165 in-domain against an alert threshold of 1.027, then 1.491 for a half-new window and 3.263 for an all-new window. Distribution shift is visible in aggregate long before any single request looks suspicious. Meanwhile, 13 of 14 out-of-scope questions got confident wrong citations: the service needs an "I don't know" path, and the monitor is what tells you to build it.

Safety: 0 side effects from 5 attacks, but 2 attacks were "answered" through the notes path (the router sent them to retrieval, which simply returned notes). That is harmless here and worth a test anyway.

Compute: tool calls cost about 640 ms each on the CPU; the notes path costs 2 ms. A GPU would cut the tool latency by a large factor and allow a bigger model, which would probably raise tool accuracy above 0.50 and make attacks *more* dangerous, because a capable model follows injected instructions.

Block 4: Self-check (20 min)

1. Why write gates before the run? *So the threshold cannot be moved to fit the result.*
2. Why does the per-query low-confidence flag fail? *In-domain and new-topic top scores overlap, so one threshold flags about the same share of both (8% vs 8%).*
3. Why is 0.60 after iteration 2 weak evidence? *It was tuned on the gate's own eval set.*
4. Why keep a test that asserts a bug? *It documents the known behaviour and forces a deliberate change when fixed.*

Block 5: Exercise (45 min)

Fix the router (e.g. require a digit or operator near "compute"), update the known-bug test, rerun the gates, and add an "I don't know" answer when the window monitor alerts *and* the top score is low. Report recall on a fresh set of 10 questions you write first.

Month-5 checkpoint exam (15 questions; 60 minutes; covers Days 121-150)

1. Write the Bellman expectation equation for V^pi and say what changes in the optimality equation.
2. Why does value iteration converge for gamma < 1?
3. Q-learning vs SARSA: which is off-policy, and why does SARSA learn a safer path on a cliff?
4. What two tricks stabilise DQN, and what failure does each address?
5. In REINFORCE, why does subtracting a baseline not bias the gradient?
6. What does PPO's clipped objective prevent?
7. Give one example of reward hacking and one defence.
8. In RLHF with PPO, what is the KL penalty against, and why is it there? How does DPO avoid a reward model?
9. UCB vs Thompson sampling: how does each balance exploration?
10. Why validate an agent's tool call against a JSON schema before executing it?
11. Define pass^k for agents and why it is stricter than pass@k.
12. What is indirect prompt injection, and which defence does not depend on detecting it?
13. Name three things that can cause training/serving skew.
14. What does PSI measure, and why choose its alert threshold from resampled reference windows?
15. A prompt change raises the pass rate but fails the regression gate. What do you do?

Exam answers

1. V^pi(s) = Sum_a pi(a|s) Sum_{s'} P(s'|s,a)[r + gammaV^pi(s')]. The optimality equation replaces the average over pi with a max over actions.
2. The Bellman optimality operator is a gamma-contraction in the max norm, so repeated application converges to its unique fixed point.
3. Q-learning is off-policy (its target uses max over next actions); SARSA uses the action actually taken, including exploratory ones, so it values cliff-edge states lower under eps-greedy and learns the safer path.
4. A replay buffer (breaks correlation between consecutive samples) and a target network (stops the target moving with every update).
5. The expected score-function gradient of a baseline that does not depend on the action is zero, since Sum_a ?pi(a|s) = 0.
6. Policy updates that move the probability ratio far from 1 in one step, which destabilises training.
7. E.g. an agent looping to collect a shaping bonus instead of finishing; defence: potential-based shaping, or reward only the true goal and audit behaviour.
8. Against the reference (SFT) policy, to stop the policy drifting into reward-model exploits and losing fluency. DPO optimises a closed-form loss on preference pairs that implicitly defines the reward via the policy/reference log-ratio.
9. UCB adds an optimism bonus that shrinks with visit count; Thompson samples from the posterior and acts greedily on the sample.
10. Model outputs are untrusted; validation catches malformed or out-of-range arguments before any side effect.
11. pass^k: the task succeeds in all k independent trials; it measures reliability, while pass@k (any of k) measures capability.
12. Instructions hidden in retrieved or tool content that the model follows. An allow-list plus human approval for side-effect actions works without detection.
13. Different preprocessing code, feature staleness or leakage, different library versions or dtypes (any three).
14. Shift between binned distributions; resampling in-domain windows gives the natural PSI spread for that window size, so the threshold has a known false-alarm rate (Day 150: 99th percentile, 1.027).

15. Read the per-case diff; decide whether the regressions matter; fix the prompt or explicitly accept and document; grow the eval set; never just ship on the aggregate.

Extra notes

A capstone that passes every gate on the first run usually has gates that are too loose. The useful output here is the list of failures with a test attached to each.

Going deeper

Compare PSI with a two-sample KS test on the same windows, and think about which one alerts sooner for a slow drift.

Interview-style question (15 min)

"Your RAG service answers out-of-scope questions confidently. How do you detect and handle it?" *Expected: window-level drift on retrieval scores and query embeddings; an abstain path; an out-of-scope eval slice; do not rely on a single per-query score threshold.*

Common mistakes

- Moving a gate after seeing the result.
- Tuning repeatedly on the gate set.
- Per-request thresholds as the only drift detection.
- No test for known bugs.

What to do next

Month 6 starts tomorrow: performance engineering, beginning with profiling the retrieval path you just built.

Day 151: Profiling Python ML code

Review today: Day 150 (+1), Day 148 (+3), Day 144 (+7), Day 130 (+21; quick 15-min skim of its self-check).

Block 1: Measure first (30 min)

Month 6 is about making code fast, and the first rule is that you do not know where the time goes until you measure. Intuition about hot spots is wrong often enough that guessing is a waste of effort. The tools, from coarse to fine: - **Wall-clock timing** with `time.perf_counter`, a warm-up call, several repeats, and the *minimum* (or median) reported. The minimum is the least noisy estimate of the code's cost; the median tells you what users see. - **cProfile** (deterministic function-level profiler in the standard library): counts calls and time per function. It adds overhead to every call, so it distorts code that makes many tiny calls, but the *ranking* is usually right. - **tracemalloc**: Python-level memory allocations, current and peak. - **torch.profiler**: per-operator times for PyTorch, including the C++ kernels beneath `nn.Linear`. - Sampling profilers (such as py-spy) and hardware counters (perf) are the next step; neither is used here.

The target is the Day 150 capstone's notes path, the Week 17 `BM25.scores`, which is pure Python.

Block 2: The run (60 min)

`day151_profiling.py` times the baseline, profiles it, measures memory, applies two fixes the profile suggests, checks the scores did not change, and profiles a small MLP.

Verified (out/day151_profiling.log): - Corpus 1,099 chunks, 58 queries; BM25 build 0.03 s. - Baseline: 1.46 ms/query (best of 3 repeats over 58 queries). - cProfile: 438,322 function calls in 0.167 s. The generator expression at `_rag.py:56` has 218,928 calls and 0.101 s own time; `sum` 63,742 calls, 0.030 s; `dict.get` 155,186 calls, 0.015 s; `torch.tensor` 0.006 s; `re.findall` under 0.001 s. - tracemalloc: building BM25 peaks at 6.1 MB (per-document Counters); one query peaks at 32 kB. - Fix A, inverted index: 0.295 ms/query (5x). Fix B, scipy sparse matrix-vector product: 0.043 ms/query (34x). Max score difference from baseline 1.8e-06 for both. W is 1,099 x 7,006 with 77,359 non-zeros (1.00% dense). - torch.profiler, MLP 384-1024-384 on batch 256, 20 calls, 1 thread: `aten::addmm` 86.03% of self CPU (41.519 ms over 40 calls), `aten::linear` 5.52%, `aten::gelu` 5.03%, `aten::copy_` 2.52%.

Block 3: Reading a profile (45 min)

The profile names the algorithm, not just the line. The hot generator is the inner loop of `scores`, which runs over *every document* for *every query*: 1,099 documents x 58 queries, with a dictionary lookup per query term. Making that loop slightly faster would help a little. The profile instead says the structure is wrong: most documents contain none of the query's words, so scanning them is wasted work.

Fix A changes the algorithm. An inverted index stores, for each term, the documents that contain it and their precomputed BM25 weight. A query then touches only the posting lists of its terms. That alone gave 5x, still in pure Python.

Fix B changes the representation. The same weights as a sparse matrix W (documents x terms) turn scoring into one sparse matrix-vector product, W.q, where q counts the query terms. The loop now runs in compiled code. 34x. The scores agree to 1.8e-06, which is float32 rounding, so the fix is correct.

Profiler overhead is real. The baseline at 1.46 ms x 58 queries is about 0.085 s of work, while cProfile reported 0.167 s for the same calls: roughly 2x inflation, because the profiler hooks every one of the 438,322 calls. Use cProfile to find *where*, and plain timing to measure *how much*.

Memory is not this code's problem. A 6.1 MB build peak and 32 kB per query are tiny. Recording that is still worth it: you now know memory is not the reason to change this code.

For the MLP, the profile says "leave it". 86% of the time is in `addmm`, the matrix multiply, which is already OpenBLAS. GELU is 5%. Fusing GELU into the matmul could save at most that 5% (Amdahl's law). To make this MLP faster you would change the matmul: fewer FLOPs (smaller layers, sparsity), a cheaper dtype (Day 152 measures bf16), or more threads.

Amdahl's law in one line: if a fraction p of the time is sped up by s, the overall speed-up is 1 / ((1 - p) + p/s). With p = 0.05, even infinite s gives only 1.05x.

Block 4: Self-check (20 min)

1. Why report the minimum of repeats? *It is the least noisy estimate; noise only adds time.*
2. Why was cProfile's total 2x the timed cost? *Per-call hook overhead on 438k calls.*
3. Why did the inverted index help? *It skips documents without query terms instead of scanning all 1,099.*
4. What is the maximum gain from fusing GELU in the MLP? *About 5% of the time; addmm dominates.*

Block 5: Exercise (45 min)

Profile the Week 17 dense-embedding retrieval (or any model forward pass you have) with torch.profiler, record the top three operators, and compute the Amdahl bound for speeding up the second one by 10x.

Extra notes

Always check correctness after an optimisation, with a tolerance that matches the dtype. The 1.8e-06 difference here is expected for float32 sums done in a different order; an exact-equality test would have failed for a correct fix.

Going deeper

Read the `cProfile` and `tracemalloc` docs (linked in `docs/resources.md`) and try `pstats` sorting by cumulative time to see callers rather than callees.

Interview-style question (15 min)

"An endpoint is slow. Walk me through how you would find out why." *Expected: reproduce with a fixed input; time end to end; trace spans to find the slow stage; profile that stage; check algorithm before micro-optimising; verify the fix keeps outputs equal; re-measure under realistic load.*

Common mistakes

- Optimising without a profile.
- Reporting cProfile times as real costs.
- Timing the first call (imports, caches, JIT, lazy loading).

- Micro-optimising a loop that should not exist.

What to do next

Tomorrow: why compiled code is fast and where the hardware limits are, with memory bandwidth, the roofline model and memory layout.

Day 152: Memory bandwidth, the roofline model, layout and fusion

Review today: Day 151 (+1), Day 149 (+3), Day 145 (+7), Day 131 (+21; quick 15-min skim of its self-check).

Block 1: Two limits (30 min)

Every kernel is limited by one of two things: how fast the CPU can do arithmetic (peak FLOP/s) or how fast memory can deliver bytes (bandwidth, GB/s). **Arithmetic intensity** is FLOPs per byte moved. The **roofline model** says attainable performance is min(peak FLOP/s, intensity x bandwidth). The *ridge point*, peak ÷ bandwidth, is the intensity where a kernel stops being memory-bound. Most of deep learning outside the big matmuls (element-wise ops, norms, softmax, embedding lookups, small-batch inference) is memory-bound, and that is why fusion and lower-precision storage matter.

Block 2: The measurements (60 min)

`day152_roofline.py` measures bandwidth, matmul throughput by dtype and thread count, places kernels on the roofline, and times layout and fusion effects. A first run crashed in section 5 (an augmented assignment `y += ...` inside a function made `y` local: `UnboundLocalError`); the fix uses `np.add(..., out=y)` chains. All numbers below are from the clean re-run.

Verified (out/day152_roofline.log): - Bandwidth, 256 MB float64, 1 thread: copy 29.5 GB/s (read + write), sum 13.3 GB/s, a+a->b 17.5 GB/s (0.73 GFLOP/s). - Matmul GFLOP/s (torch, nxn), 1 thread at n=64/256/1024/2048: float64 87.0/110.6/114.4/110.9; float32 74.3/226.7/235.4/238.6; bfloat16 36.8/862.1/1654.4/1581.5. - 8 threads: float64 57.0/375.8/436.0/701.1; float32 74.2/1040.6/1666.2/1701.1; bfloat16 29.9/1450.1/8826.9/11843.7. - Roofline, float32, 1 thread: peak 239 GFLOP/s, bandwidth 29.5 GB/s -> ridge 8.1 FLOP/byte. Vector add 0.08 FLOP/byte (<= 2.5 GFLOP/s); dot 0.25 (<= 7.4); matvec 0.50 (<= 14.8); matmul n=1024 170.67 (compute-bound, <= 238.6). - Measured matvec n=4096 float32: 17.1 GFLOP/s, 34.3 GB/s of matrix read, *above* the DRAM roof, because the 64 MB matrix fits in the 320 MB L3 cache. - Layout, 4096x4096 float64: sum(axis=1) 4.8 ms, sum(axis=0) 4.5 ms; 512 row slices 0.8 ms vs 512 column slices 13.0 ms (15.8x slower, strided); making the transpose contiguous 128.2 ms (2.1 GB/s). - Fusion: 2x^2+3x+1 on 8M float64: naive (temporaries) 26.9 ms; in-place Horner with `out=` 12.1 ms (2.2x); max difference 1.8e-15. - Dot product of 1M floats: Python loop 30.4 ms, numpy 0.037 ms (832x).

Block 3: What it means (45 min)

Element-wise work is starved. Adding two arrays reaches 0.73 GFLOP/s, about 0.3% of the 239 GFLOP/s single-thread float32 matmul peak. The arithmetic is free; the bytes are not. The roofline predicts this: at 0.08 FLOP/byte the ceiling is 2.5 GFLOP/s even with perfect code.

Matmul is the opposite. At n=1024, intensity is 170 FLOP/byte, far right of the 8.1 ridge, and float32 reaches 235-239 GFLOP/s on one thread, the measured peak. Small matrices are the exception: at n=64, multi-threading *hurts* (float64 87.0 -> 57.0) because thread start-up costs more than the work.

bf16 and AMX. This Xeon has AMX matrix tiles. bf16 matmul on 8 threads reaches 11,843.7 GFLOP/s at n=2048, about 7x float32's 1,701.1. At n=64 it is the slowest option (29.9-36.8), since converting and tiling dominate. Lower precision pays only when the matmul is big enough to keep the tiles busy. On a GPU the same lesson holds with tensor cores.

Caches bend the roofline. The matvec measured 17.1 GFLOP/s against a 14.8 DRAM-bound prediction, because the matrix sat in L3. A real roofline has one memory roof per cache level. Roofline numbers are upper bounds for a given level of the hierarchy, not exact predictions.

Layout decides bandwidth. NumPy arrays are row-major (C order) by default. A row slice is contiguous; a column slice reads one element per 32 KB row stride, wasting most of each cache line: 15.8x slower. Whole-array reductions along either axis were close (4.8 vs 4.5 ms) because numpy reorders the loops internally. Making a transpose contiguous costs a full copy at only 2.1

GB/s, since the strided reads are the slow part.

Fusion saves passes over memory. The naive polynomial makes several temporary 64 MB arrays, each a full read-and-write pass. The `out=` version reuses one buffer, halving the time with identical results. Compilers for ML (XLA, TorchInductor, Triton kernels) do this automatically by fusing element-wise chains into one loop.

Python loops are the worst case. 832x for a dot product: each Python iteration is an interpreted bytecode dispatch plus boxed float objects.

GPU note (not measured): a datacenter GPU has much higher bandwidth and far higher peak FLOP/s, so its ridge point sits at a similar or higher intensity. Element-wise kernels remain memory-bound there too, which is why the fused-softmax idea shown in Day 154's UNTESTED Triton file matters.

Block 4: Self-check (20 min)

1. Compute the dot product's intensity in float32. *2 FLOPs per 8 bytes = 0.25 FLOP/byte.*
2. Why did 8 threads slow down n=64 float64 matmul? *Threading overhead exceeds the tiny amount of work.*
3. Why can measured matvec beat the DRAM roof? *The matrix was in L3 cache, which has higher bandwidth.*
4. Why are column slices of a C-order array slow? *Strided access uses one element per cache line.*

Block 5: Exercise (45 min)

Re-run section 1 with 8 threads (`torch.set_num_threads(8)` and a threaded numpy op) and see how much bandwidth rises. Then compute where a LayerNorm over a 4096x1024 float32 tensor sits on the roofline.

Extra notes

Always state the dtype, thread count and size with any throughput number. "1,700 GFLOP/s" means nothing without "float32, 8 threads, n=2048".

Going deeper

Read about the cache-aware roofline model and about how blocked (tiled) matmul raises the intensity seen by each cache level.

Interview-style question (15 min)

"Your model's inference is half element-wise ops by time. What do you try?" *Expected: they are memory-bound, so fuse them (torch.compile or custom kernels), reduce bytes (bf16/int8 storage), avoid layout copies, batch to raise intensity.*

Common mistakes

- Counting FLOPs without bytes.
- Expecting threads to help small kernels.
- Ignoring hidden copies from transposes and `.contiguous()`.
- Comparing dtypes at a size where conversion dominates.

What to do next

Tomorrow you write your own compiled kernels in Rust and call them from Python, testing vectorisation and threading against the limits you measured today.

Day 153: Rust extensions with PyO3: vectorisation and threads

Review today: Day 152 (+1), Day 150 (+3), Day 146 (+7), Day 132 (+21; quick 15-min skim of its self-check).

Block 1: Why Rust, and how the binding works (30 min)

When numpy has no single call for what you need, Python loops are 800x too slow (Day 152), so you write the kernel in a compiled language. The syllabus suggests C++ with pybind11; this box has no C++ compiler, so the course uses Rust, which has

three advantages for this job: memory safety without a garbage collector, a package manager (cargo) that builds with one command, and **PyO3**, which generates the Python binding from annotated Rust functions. **maturin** builds the crate into a Python wheel and installs it into the venv (`maturin develop --release`). The `numpy` crate gives zero-copy read access to numpy arrays as Rust slices, and `py.allow_threads` releases Python's GIL while the kernel runs so other Python threads can continue.

The crate `rust_kernels/src/lib.rs` has three families: - **dot products:** `dot_naive` (one accumulator, indexed loop), `dot_simd` (16 independent accumulators over `chunks_exact(16)`), `dot_par` (rayon splits the array into 64K-element chunks over 8 threads, each using the SIMD kernel). - **row softmax:** a 3-pass version (max; exp and sum; scale) and an "online" version (running max and rescaled running sum in one read pass, then one write pass), each serial or rayon-parallel over rows. - **BM25 top-k** over a sparse matrix, used tomorrow.

Why the naive dot cannot vectorise. Floating-point addition is not associative, so the compiler must add the products in exactly the order written: one long dependency chain, one addition at a time. The 16-accumulator version gives the compiler 16 independent sums it may place in SIMD lanes, and `chunks_exact` removes bounds checks from the loop. Rust will not reorder float sums on its own (there is no equivalent of `-ffast-math` by default), so you write the reordering yourself.

Block 2: The run (60 min)

`day153_rust_kernels.py` builds the crate twice with maturin, once for portable x86-64 and once with `RUSTFLAGS="-C target-cpu=native"` (allowing AVX-512 on this CPU), and benchmarks each build in a fresh process. numpy is pinned to one BLAS thread; rayon uses 8.

Verified (out/day153_rust_kernels.log): - Builds: portable exit 0 in 1 s (cached), native exit 0 in 8 s. - Portable build, dot product, µs (GFLOP/s): - n=65,536 (0.5 MB): numpy 6.2 (21.1); naive 35.2 (3.7); simd 6.3 (20.7); rayon 6.5 (20.1). - n=1,048,576 (8 MB): numpy 248.5 (8.4); naive 528.7 (4.0); simd 233.5 (9.0); rayon 42.7 (49.1). - n=16,777,216 (128 MB): numpy 3,972.0 (8.4); naive 8,492.7 (4.0); simd 3,684.9 (9.1); rayon 520.4 (64.5). - Native build: simd 5.4 µs (24.4) at 0.5 MB; 253.4 µs (8.3) at 8 MB; 4,004.1 µs (8.4) at 128 MB; rayon 38.6 µs (54.3) at 8 MB and 567.0 µs (59.2) at 128 MB; naive unchanged (4.0 GFLOP/s). - Relative error vs a float64 reference at 128 MB: naive 5.3e-05, simd 4.3e-06, numpy 7.4e-06. - Softmax 4096x1024 float32, portable build: numpy 7.72 ms; rust 3-pass 10.38 ms; rust online 18.70 ms; 3-pass rayon 2.07 ms; online rayon 3.46 ms. Native: numpy 7.82; 3-pass 9.87; online 18.90; 3-pass rayon 1.83; online rayon 3.14. Max error vs float64: numpy 3.1e-07, all Rust versions 4.7e-06.

Block 3: Reading it against the roofline (45 min)

Accumulators matter more than compiler flags. The single-accumulator loop runs at 3.7-4.0 GFLOP/s at every size and with either build: it is latency-bound on its dependency chain, not memory-bound. Sixteen accumulators give 5-6x (20.7 vs 3.7 at 0.5 MB), equal to numpy's hand-tuned BLAS. `target-cpu=native` added a little only when the data sat in L2 (20.7 -> 24.4 GFLOP/s at 0.5 MB) and nothing at larger sizes.

Once data leaves the core's cache, one thread hits the memory roof. At 8 MB and 128 MB, numpy and the SIMD kernel both sit at 8.3-9.1 GFLOP/s, which at 8 bytes per 2 FLOPs is about 34-36 GB/s, the same range as Day 152's single-thread bandwidth. No amount of SIMD helps past that. **Threads** add memory parallelism: rayon reached 49-64.5 GFLOP/s, about 6-8x one thread. (Both large arrays fit in the 320 MB L3, so this is L3 bandwidth across cores, not DRAM.) At 0.5 MB, rayon gave nothing, since the work is only microseconds and threading overhead eats the gain.

Accuracy improves with more accumulators. The naive sum's relative error (5.3e-05 at 16M elements) is about 12x the SIMD version's (4.3e-06): 16 partial sums each add fewer terms, which acts like a partial pairwise summation. Faster and more accurate here.

The online softmax lost, and that is the honest lesson. In 1-D it reads the input once instead of twice, but it computes `exp` twice per element (once in the running sum, once for the output), plus a data-dependent branch. Each row is 4 KB, which stays in L1 cache, so the saved memory pass was nearly free anyway; the extra `exp` work is what showed up, 18.7 vs 10.4 ms. Online softmax pays off when memory traffic is the real cost: in FlashAttention on a GPU, where it lets attention be computed in tiles without writing the full score matrix to slow memory.

numpy's softmax beat serial Rust (7.72 vs 10.38 ms) because numpy's `exp` is a vectorised SIMD implementation, while Rust's `f32::exp` calls the scalar libm routine. Rayon over rows was 3.7-4.3x faster than numpy. The fair comparison would be numpy with threads too, but numpy's element-wise ufuncs are single-threaded.

CPU vs GPU. A GPU runs this softmax with one thread block per row and thousands of rows at once; the UNTESTED Triton file in Day 154 shows that form. Nothing here measured a GPU.

Block 4: Self-check (20 min)

1. Why does `-C target-cpu=native` not speed up `dot_naive`? *Its limit is the serial add chain, which no instruction set removes.*
2. Why do numpy and `dot_simd` tie at 128 MB? *Both are bandwidth-bound at about 34 GB/s on one thread.*
3. Why was online softmax slower? *Twice the exp work, and the saved pass was over L1-resident data.*
4. What does `py.allow_threads` do? *It releases the GIL so other Python threads run during the kernel.*

Block 5: Exercise (45 min)

Add a `saxpy` (y = a.x + y, in place on a writable numpy array) kernel, predict its GFLOP/s from Day 152's roofline before running, then measure the serial and rayon versions at 1M and 16M elements.

Extra notes

Correctness tests come first: every kernel here was compared with a float64 numpy reference before timing. Benchmarks used the minimum of 7 runs after a warm-up, in a fresh process per build so the two builds cannot mix.

Going deeper

The PyO3 guide's sections on the GIL and on numpy interop, and the rayon docs on work stealing (all linked in `docs/resources.md`).

Interview-style question (15 min)

"You rewrote a numpy function in Rust and it is slower. Why might that be?" *Expected: numpy was already calling vectorised C/BLAS; scalar math functions in Rust; bounds checks or a serial dependency chain; copies at the Python boundary; measuring the first call; memory-bound so no compute gain possible.*

Common mistakes

- Expecting compiled code to beat BLAS by default.
- One accumulator in a reduction.
- Benchmarking threads on microsecond-sized work.
- Comparing builds in the same process.

What to do next

Tomorrow's project uses the BM25 kernel to speed up the capstone's retrieval end to end, and shows GPU kernels as UNTESTED reading material.

Day 154: PROJECT W22: a Rust kernel for the capstone's retrieval, plus GPU kernels (UNTESTED)

Review today: Day 153 (+1), Day 151 (+3), Day 147 (+7), Day 133 (+21; quick 15-min skim of its self-check).

Block 1: The plan (30 min)

SYLLABUS Day 154 asks for a C++ pybind11 custom op. With no g++ on the box, the project does the same job in Rust: take the bottleneck found on Day 151 (BM25 scoring in Python), move it into a compiled kernel with a minimal Python interface, keep the results identical, and measure the speed-up end to end, including the parts you did not optimise.

The kernel `bm25_topk` takes scipy's CSC (term-major) arrays of the BM25 weight matrix, a list of token-id lists for the queries, and k. For each query it adds the weight column of each query term into a score buffer (touching only documents that contain the term, the inverted-index idea from Day 151), then takes the top k with `select_nth_unstable` (linear time) and sorts those k by score, breaking ties by document id. With `parallel=True`, rayon scores different queries on different threads.

Block 2: The run (60 min)

`day154_project.py` checks agreement with the Week 17 Python baseline, times each stage, then tiles the corpus 64x to see scaling.

Verified (out/day154_project.log): - Corpus 1,099 chunks x 7,006 terms, 77,359 non-zeros; 58 queries; top-5. - Agreement with the Python baseline (identical ranked top-5 lists): Rust 58/58, scipy 58/58. - Per query (ms): tokenise 0.0026; Python score + sort 1.513; scipy 0.0743; Rust 0.0044; Rust rayon 0.0019. - End to end with Rust (tokenise + kernel) 0.0070 ms, 216x faster than Python; tokenising is now 37% of the time; Amdahl's cap with an infinitely fast kernel is 591x. - 64x tiled corpus: 70,336 chunks, 4,950,976 non-zeros (39.6 MB); top-5 score sets match scipy 58/58. - Per query (ms): scipy 3.089; Rust 0.245; Rust rayon over queries (8 threads) 0.036, 6.8x over serial Rust.

Block 3: Analysis (45 min)

Correct first. All 58 ranked top-5 lists match the Python baseline exactly, including order. On the tiled corpus every document appears 64 times, so ties are everywhere; there the check compares the *score sets* of the top 5, which all match.

Why the Rust kernel beats scipy by 17x on the real corpus. Day 151's sparse product multiplied the full matrix by a dense 7,006-long count vector, which touches all 77,359 non-zeros for every query, plus Python overhead for building that vector and for `argpartition`. The Rust kernel walks only the posting lists of the query's own terms. Same maths, less memory traffic: the roofline lesson applied to sparse data. On the 64x corpus the gap is 12.6x (3.089 vs 0.245 ms).

Amdahl after the fix. Once scoring costs 0.0044 ms, the Python tokeniser (0.0026 ms, a single regex) is 37% of the end-to-end time. Speeding the kernel up further can never give more than 591x over the baseline. The next fix would be tokenising in Rust as well, or batching so per-call overheads are paid once.

When threads help. On the small corpus rayon cut 0.0044 to 0.0019 ms per query, but these are microsecond tasks and the gain is only 2.3x. With 64x more data, each query is real work and 8 threads gave 6.8x, close to linear. Parallelising across independent queries (not inside one query) is the simple, contention-free choice.

Does it matter for the capstone? Honestly, not yet: the capstone's notes path was 1.8 ms at the median, and the tool path 643 ms is the model call. The Rust kernel matters when the corpus is millions of chunks or when retrieval is called many times per request (re-ranking loops, agents calling search repeatedly). Performance work should follow the trace, and the Day 150 trace says the LLM call dominates.

Block 4: GPU kernels, UNTESTED (20 min)

There is no GPU, nvcc or Triton here, so these two files were **never compiled or run**. Read them; do not cite any numbers for them. - `gpu_untested/softmax_triton_UNTESTED.py`: one Triton program per row loads the whole row once with a mask, subtracts the max, exponentiates, divides by the sum and stores once: a fused, single-read, single-write softmax. Its `__main__` block shows the check you would run on a GPU (`torch.testing.assert_close` against `torch.softmax`). - `gpu_untested/vector_add_UNTESTED.cu`: the CUDA "hello world": one thread per element, `blockIdx.x * blockDim.x + threadIdx.x` indexing, neighbouring threads reading neighbouring addresses (coalesced loads), a bounds check, and error checking after `cudaDeviceSynchronize`.

On a GPU you would measure both against the bandwidth roof exactly as Day 152 did on the CPU: vector add moves 12 bytes per FLOP and will be bandwidth-bound there too.

Block 5: Exercise (45 min)

Move tokenisation into Rust (lowercase plus split on non-alphanumerics, matching `[a-z0-9]+`), verify identical token ids on all 58 queries, and recompute the end-to-end time and the new Amdahl split.

Extra notes

A compiled kernel is a maintenance cost: a second language, a build step (maturin) and platform-specific wheels. Use one when the profile shows a hot loop that numpy or scipy cannot express efficiently, as here, not by default.

Going deeper

Work through the Triton fused-softmax tutorial and the CUDA C++ Programming Guide's chapter on the programming model (both linked in `docs/resources.md`), then compare their thread-per-row design with this week's rayon-per-row CPU version.

Interview-style question (15 min)

"How would you decide whether to write a custom kernel for part of a model or a service?" *Expected: profile shows it dominates; no library op covers it; estimate the roofline bound and Amdahl gain first; keep a reference implementation and a correctness test; benchmark end to end; account for build and maintenance cost.*

Common mistakes

- Comparing kernels on different outputs (top-k ties).
- Reporting kernel speed-up instead of end-to-end speed-up.
- Parallelising microsecond work.
- Citing numbers for code that never ran on a GPU.

What to do next

Week 23 continues Month 6 per SYLLABUS: the GPU programming model and CUDA, Triton kernels and FlashAttention ideas (reading-only and UNTESTED on this box unless a GPU becomes available), distributed-training basics, ONNX Runtime and on-device ML.

Weekly quiz: Week 22 (12 questions; 40 minutes; covers Days 148-154)

1. On Day 148, v3 scored 13/24 and v2 7/24. Why did the gate fail, and which cases broke?
2. What makes a generation cache safe to reuse, and what was checked on Day 148?
3. On Day 149, what share of the assumed cost came from input tokens, and what is the cheapest fix?
4. Of 8 attacks "blocked" on Day 149, how many were genuinely recognised as attacks, and why does the answer matter?
5. Why did the Day 150 release get blocked?
6. Why did the per-query low-confidence flag fail on Day 150 while PSI worked? Give the PSI values.
7. What did cProfile say about BM25 on Day 151, and what were the two fixes' speed-ups?
8. Compute the arithmetic intensity of float32 vector add and its roofline ceiling from Day 152's bandwidth.
9. Why was bf16 the slowest dtype at n=64 but about 7x float32 at n=2048?
10. Why does a 16-accumulator dot product run 5-6x faster than a single-accumulator one, and why does it stop helping at 128 MB?
11. Why was the online softmax slower than the 3-pass one on the CPU?
12. In the Day 154 project, what limits further end-to-end speed-up, and by how much?

Answers

1. It lost 3 seen cases v2 passed (`1024/16`, `99/3`, `45%7`); the gate forbids per-case regressions whatever the aggregate (McNemar p=0.146).
2. Deterministic decoding and a key covering prompt hash, input, model and settings; a rerun gave 24 hits and identical results, and greedy outputs were identical when generated twice.
3. 85% (mean 421 input vs 24 output tokens); shorten the few-shot prompt or cache the prefix.
4. 6 by the input regex; the 2 output blocks were schema failures. The model never called `send_email` anyway, and 4 paraphrases evaded the regex, so the regex is weak protection.
5. recall@3 was 0.53 (< 0.60) and one notes question was misrouted to tools by the word "compute".
6. Top scores overlap: 8% flagged in-domain and 8% for all-new topics. Window PSI: 0.165 in-domain vs threshold 1.027; 1.491 half-new; 3.263 all-new.
7. The inner generator scanning every document dominated (0.101 of 0.167 s); inverted index 5x, sparse mat-vec 34x.
8. 1 FLOP per 12 bytes ~= 0.08 FLOP/byte; 0.08 x 29.5 GB/s ~= 2.5 GFLOP/s.
9. At n=64 conversion and tiling overhead dominate (29.9-36.8 GFLOP/s); at n=2048 AMX tiles are busy: 11,843.7 vs 1,701.1 GFLOP/s on 8 threads.
10. Independent sums break the serial add chain and fill SIMD lanes; at 128 MB one thread is bandwidth-bound (~8.4 GFLOP/s for numpy and SIMD alike); only threads (59-64.5 GFLOP/s) helped.
11. It computes exp twice per element plus a branch, and the pass it saves is over L1-resident 4 KB rows.

12. Tokenising (37% of the 0.0070 ms); Amdahl caps the gain at 591x over Python even with an infinitely fast kernel.